

Server side Programming 2

Luxe-Living Laravel Website

Security Documentation and Implementation

Table of Content

1. Authentication.....	2
1.1. Laravel Sanctum.....	2
1.2. Laravel Jetstream.....	3
1.3. OAuth Integration.....	3
1.4. 2 Factor Authentication.....	4
2. Authorization.....	5
2.1. Role based access control.....	5
3. Request Protection.....	7
3.1. CSRF Tokens.....	7
4. Password Protection.....	8
5. SQL Injection Prevention.....	8
6. Blade Template Security.....	8
7. Use of NoSQL Database.....	9
8. Hosting.....	10

1. Authentication

1.1. Laravel Sanctum

Purpose: It provides lightweight API authentication for SPA, mobile apps and token based APIs.

Implementation:

- Installation - composer require laravel/sanctum

```
"laravel/sanctum": "^4.2",
```

- Configured middleware for protected routes

```
// Protected routes - Customer
Route::middleware(['auth:sanctum', 'rolemanager:customer'])->group(function () {
    // User profile
    Route::get('/user/profile', [AuthController::class, 'profile']);
    Route::put('/user/profile', [AuthController::class, 'updateProfile']);
});
```

```
// Protected routes - Admin
Route::middleware(['auth:sanctum', 'rolemanager:admin'])->group(function () {
    Route::prefix('admin')->group(function () {
        // Dashboard stats
        Route::get('/dashboard', [AdminMainController::class, 'dashboard']);
    });
});
```

- Token Management is stored in the personal_access_tokens table

id	tokenable_type	tokenable_id	name	token	abilities	last_used_at	expires_at	created_at	updated_at
1	App\Models\User	3	mobile_app	2284636f83395bebb5288b2a12cc80447679ffec8a39bdd0ab...	["*"]	2025-10-03 05:53:55	NULL	2025-10-03 05:53:53	2025-10-03 05:53:55

I have connected the APIs to my flutter mobile application. The routes are protected through Sanctum.

```
// api.php
Route::middleware(['auth:sanctum',
'rolemanager:customer'])->group(function () {
    // User profile
    Route::get('/user/profile', [AuthController::class, 'profile']);
    Route::put('/user/profile', [AuthController::class, 'updateProfile']);
});
```

1.2. Laravel Jetstream

Purpose: provides full authentication scaffolding (login, registration, email verification)

Implementation:

- Installation: composer require laravel/jetstream

```
"laravel/jetstream": "^5.3",
```

- Configured with livewire for front-end

```
Route::middleware('auth')->group(function () {
    Route::get('verify-email', EmailVerificationPromptController::class)
        ->name('verification.notice');
```

```
@if (Laravel\Jetstream\Jetstream::hasTermsAndPrivacyPolicyFeature())
    <div class="mt-4">
        <x-label for="terms">
            <div class="flex items-center">
                <x-checkbox name="terms" id="terms" required />
```

1.3. OAuth Integration

Purpose: enables login from google accounts/ third-party logins

Implementation:

- Installation: composer require laravel/socialite

```
"laravel/socialite": "^5.23",
```

- Sign in with Google link is displayed on the login form

```

use Illuminate\Support\Facades\Auth;
use Illuminate\Support\Facades\Log;
use Exception;

class GoogleAuthController extends Controller
{
    public function redirect()
    {
        return Socialite::driver('google')->redirect();
    }

    public function callback(Request $request)
    {
        try {
            if ($request->has('error')) {
                Log::error('Google Auth Error from Google: ' . $request->get('error'));
                return redirect('/login')->with('error', 'Authentication was denied.');
```

- The logged in google details will be stored in the user model

* id	name	email	avatar	role	email_verified_at	password	two_factor_secret	remember_token
1	admin1	admin@gmail.com	NULL	0	NULL	\$2y\$12\$WjyU7ueCLE3pnPWSBXEOAFxmg7Lo0hDCLRG7RohC...	NULL	NULL
2	Shalindri Sathiyathan	shalindriapiit@gmail.com	https://lh3.googleusercontent.com/taACgltocKyaQW4qb...	1	2025-10-03 05:16:44	NULL	NULL	k0vmbC8GQqWheutsasC4vEQG8YTYt4nwKvVHQVmpPc0Xm8aG8

current_team_id	profile_photo_path	created_at	updated_at	google2fa_secret	two_factor_recovery_codes	two_factor_confirmed_at	google_id	google_token	google_refresh_token
NULL	NULL	2025-10-03 04:10:53	2025-10-03 04:10:53	NULL	NULL	NULL	NULL	NULL	NULL
NULL	NULL	2025-10-03 04:25:49	2025-10-03 05:16:44	NULL	NULL	NULL	101790893108637750317	NULL	NULL

- Google Authentication routes

```

GET|HEAD      auth/google ..... google.redirect -> GoogleAuthController@redirect
GET|HEAD      auth/google/callback ..... google.callback -> GoogleAuthController@callback
```

- Logs

```

[2025-10-03 04:25:49] local.INFO: Google User Data: {"id":"101790893108637750317","email":"shalindriapiit@gmail.com","name":"Shalindri Sathiyathan"}
[2025-10-03 04:35:56] local.INFO: Google User Data: {"id":"101790893108637750317","email":"shalindriapiit@gmail.com","name":"Shalindri Sathiyathan"}
[2025-10-03 04:35:58] local.INFO: Google User Data: {"id":"101790893108637750317","email":"shalindriapiit@gmail.com","name":"Shalindri Sathiyathan"}
[2025-10-03 04:37:41] local.INFO: Cart add request {"product_id":1,"quantity":1}
[2025-10-03 04:37:50] local.INFO: Cart add request {"product_id":1,"quantity":1}
[2025-10-03 04:38:04] local.INFO: Cart update request {"_token":"meRqSi3GjpCrGD4vMPVsafCwCcmxuqyOLPVvd00","_method":"PATCH","cart_item_id":1,"quantity":1}
[2025-10-03 05:07:01] local.INFO: Google User Data: {"id":"101790893108637750317","email":"shalindriapiit@gmail.com","name":"Shalindri Sathiyathan"}
[2025-10-03 05:11:27] local.INFO: Google User Data: {"id":"101790893108637750317","email":"shalindriapiit@gmail.com","name":"Shalindri Sathiyathan"}
[2025-10-03 05:12:06] local.INFO: Google User Data: {"id":"101790893108637750317","email":"shalindriapiit@gmail.com","name":"Shalindri Sathiyathan"}
[2025-10-03 05:16:08] local.INFO: Google User Data: {"id":"101790893108637750317","email":"shalindriapiit@gmail.com","name":"Shalindri Sathiyathan"}
[2025-10-03 05:16:09] local.INFO: Google User Data: {"id":"101790893108637750317","email":"shalindriapiit@gmail.com","name":"Shalindri Sathiyathan"}
```

1.4. 2 Factor Authentication

Purpose: Adds an extra layer of security beyond passwords through Google authenticator.

Implementation:

- Jetstream's built-in 2FA via Google Authenticator

`pragmarx/google2fa-laravel`: `"^2.3"`

- 2FA Routes

```

..... password.confirmation > Laravel\Fortify > ConfirmedPasswordStatusControl
ler@show
POST      user/confirmed-two-factor-authentication .....
..... two-factor.confirm > Laravel\Fortify > ConfirmedTwoFactorAuthenticationControll
er@store
PUT       user/password .....
..... user-password.update > Laravel\Fortify > PasswordControlle
r@update
GET|HEAD  user/profile .....
..... profile.show > Laravel\Jetstream > UserProfileControl
ler@show
PUT       user/profile-information .....
..... user-profile-information.update > Laravel\Fortify > ProfileInformationControlle
r@update
POST      user/two-factor-authentication .....
..... two-factor.enable > Laravel\Fortify > TwoFactorAuthenticationControll
er@store
DELETE    user/two-factor-authentication .....
..... two-factor.disable > Laravel\Fortify > TwoFactorAuthenticationController
@destroy
GET|HEAD  user/two-factor-qr-code .....
..... two-factor.qr-code > Laravel\Fortify > TwoFactorQrCodeControl
ler@show
GET|HEAD  user/two-factor-recovery-codes .....
..... two-factor.recovery-codes > Laravel\Fortify > RecoveryCodeControl
er@index
POST      user/two-factor-recovery-codes .....
..... two-factor.regenerate-recovery-codes > Laravel\Fortify > RecoveryCodeControl
er@store
GET|HEAD  user/two-factor-secret-key .....
..... two-factor.secret-key > Laravel\Fortify > TwoFactorSecretKeyControl
ler@show
GET|HEAD  verify-email .....
DELETE    user/two-factor-authentication .....
..... two-factor.disable > Laravel\Fortify > TwoFactorAuthenticationController@

```

The authentication is partially working due to some errors

2. Authorization

2.1. Role based access control

- Admin → Admin dashboard
- Customer → User dashboard

```

public function handle(Request $request, Closure $next, $role): Response
{
    // Check if user is authenticated
    if (!Auth::check()) {
        if ($request->expectsJson()) {
            return response()->json(['message' => 'Unauthenticated.'], 401);
        }
        return redirect()->route('login');
    }
}

```

```

$authUserRole = Auth::user()->role;

switch ($role) {
    case 'admin':
        if ($authUserRole == 0) {
            return $next($request);
        }
        break;
    case 'customer':
        if ($authUserRole == 1) {
            return $next($request);
        }
        break;
}

```

```

// Redirect users according to role
switch ($authUserRole) {
    case 0:
        return redirect()->route('admin');
    case 1:
        return redirect()->route('user.user');
}

```

- Routes in web.php

```

//admin routes
Route::middleware(['auth:sanctum',
'verified','rolemanager:admin'])->group(function () {

```

```
Route::prefix('admin')->group(function () {
    Route::controller(AdminMainController::class)->group(function () {
        Route::get('/dashboard','index')->name('admin');
```

```
Route::middleware(['auth:sanctum',
'verified','rolemanager:customer'])->group(function () {
    Route::prefix('customer')->group(function () {
        Route::controller(ProductController::class)->group(function () {
            Route::get('/dashboard','index')->name('user.user'); // Home page
            Route::get('/products/{product}','show')->name('user.show');
```

3. Request Protection

3.1. CSRF Tokens

Purpose: to prevent cross-site request forgery attacks and protects forms and AJAX from CSRF attacks.

Implementation:

- Laravel automatically generates CSRF tokens for forms

```
<!-- Payment Form -->
<div class="lg:col-span-7">
    <form method="POST" action="{{ route('payments.store', $order) }}"
enctype="multipart/form-data" class="space-y-6">
        @csrf
```

- Passes Laravel's CSRF Token to the front-end

```
<head>
```

```
<meta charset="utf-8">
<meta name="viewport" content="width=device-width, initial-scale=1">
<meta name="csrf-token" content="{{ csrf_token() }}">
```

4. Password Protection

- Passwords are hashed using **bcrypt/argon2id** (Laravel default).

	email_verified_at	password
0	NULL	\$2y\$12\$WjyyU7ueCLE3pnPNWSBXEOAFXmg7LoGkDfCLRG7RohC...
1	2025-10-03 05:16:44	NULL
1	NULL	\$2y\$12\$ZXwKtE7dUWpOlr9HnOHdOuB1rw0FuWGWcwsI7sirSEU...

5. SQL Injection Prevention

- Use of Eloquent ORM – uses parameter binding

```
$category_info = Category::find($id);

$category=Category::findOrFail($id);

Category::findOrFail($id)->delete();
```

6. Blade Template Security

- Escaping user output to prevent XSS (Cross-Site Scripting) when displaying data

```
<li class="breadcrumb-item"><a href="{{ route('user.user') }}">Products</a></li>

<h1 class="display-5 fw-bold mb-3">{{ $product->product_name ?? 'Product Name' }}</h1>
```


7. Use of NoSQL Database

MongoDB Compass for Wishlist

```
//Wishlist.php
namespace App\Models;

use MongoDB\Laravel\Eloquent\Model;

class Wishlist extends Model
{
    protected $connection='mongodb';
    protected $collection='wishlist';

    protected $fillable=[
        'user_id',
        'product_id',

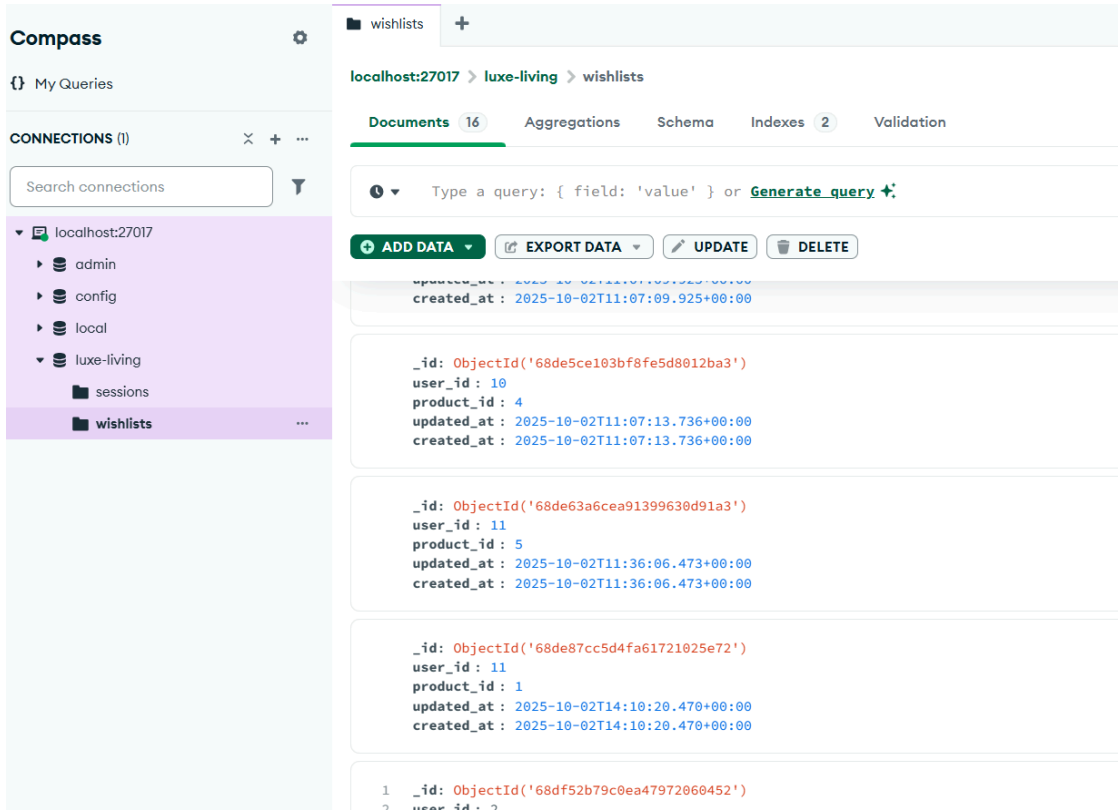
    ];

    protected $casts=[
        'user_id' => 'integer',
        'product_id' => 'integer',
        'created_at' => 'datetime',
        'updated_at' => 'datetime',
    ];

    //get product from mysql
    public function product(){
        return $this->belongsTo(\App\Models\Product::class,'product_id');
    }

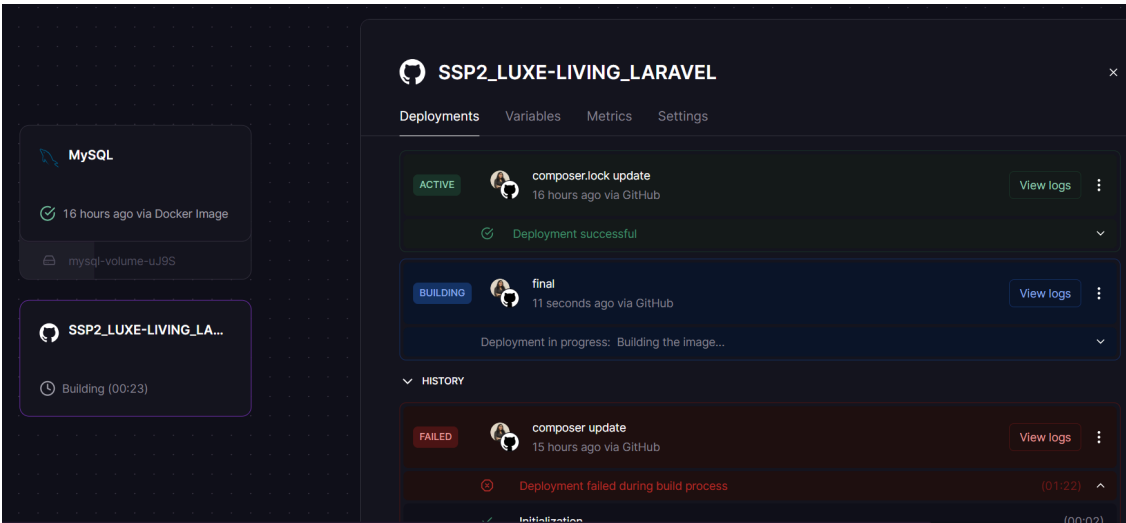
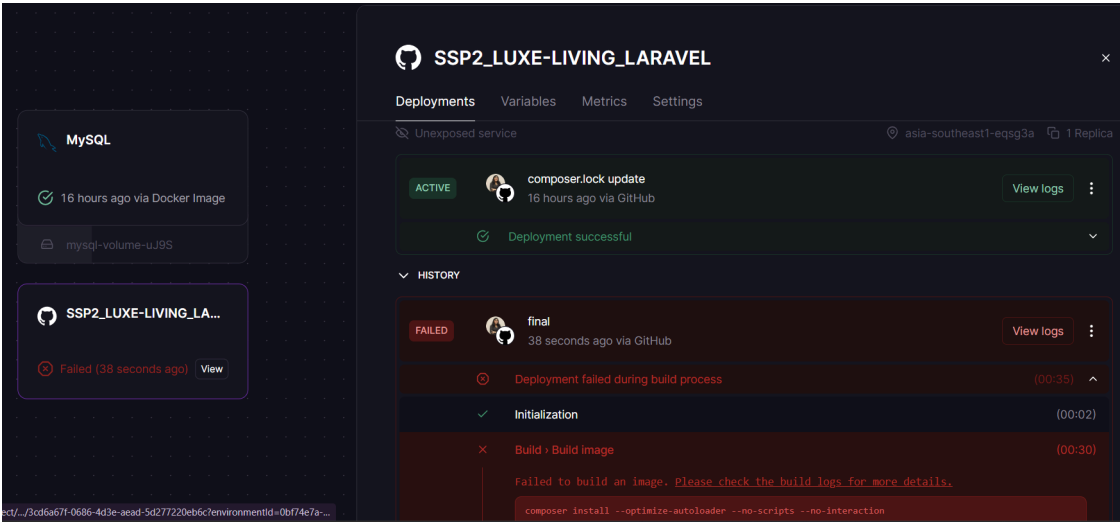
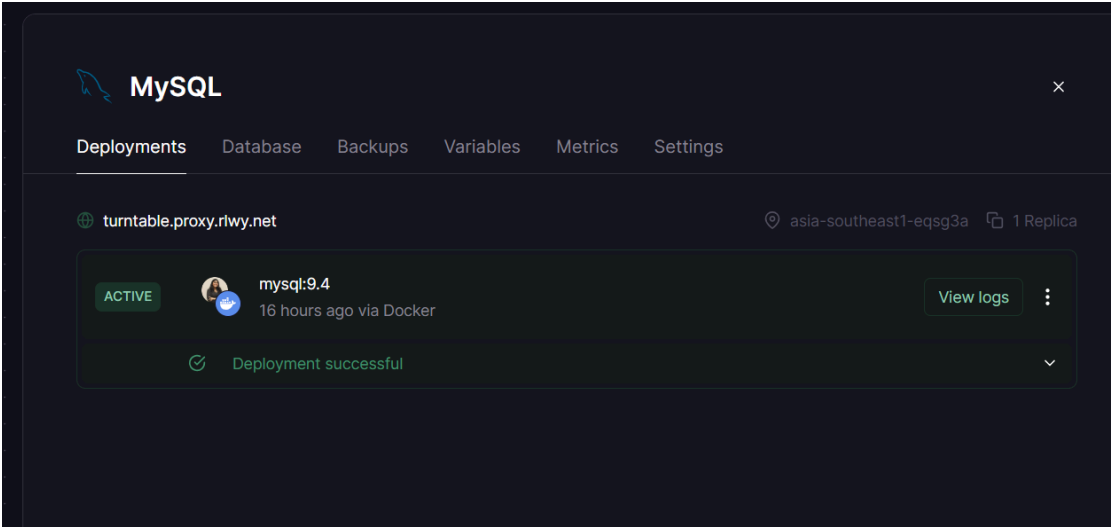
    //get user from mysql
    public function user(){
```

```
return $this->belongsTo(\App\Models\User::class,'user_id');  
}
```



8. Hosting

Status: partially completed
Reason: Due to version incompatibility.



Github repo: https://github.com/shalindri0426/SSP2_LUXE-LIVING_LARAVEL.git